Règles de conception pour les classes du modèle

Nous reprenons ci-dessous quelques **règles de bonne pratique** pour la conception de vos classes du modèle.

> **Remarque**
>
> Si, malgré ces règles, vous êtes confrontés à un cas pour lequel vous hésitez sur la nature de la méthode à créer (publique/privée, statique/instance, dans quelle classe, quelle signature ?), n'hésitez pas à poser la question à votre professeur.

Règles obligatoires

Vous prévoirez des attributs privés que vous accèderez grâce à des accesseurs publics, en lecture et/ou écriture selon les besoins.

Dans la majorité des cas, vos méthodes doivent être des **méthodes d'instance**. Les seules méthodes statiques autorisées sont :

- des méthodes privées qui effectuent les requêtes SQL et qui peuvent être appelées par plusieurs méthodes d'instances (on factorise ainsi les commandes SQL) ;
- des méthodes publiques qui font des recherches (lectures) dans la base de données. Ces méthodes doivent retourner une instance ou un tableau d'instances, sauf si l'objet de la lecture est de faire une vérification (dans ce cas on retourne un booléen) ou un comptage (dans ce cas on retourne un entier) ;
- des méthodes "utilitaires", publiques ou privées, qui n'interviennent pas sur les données de la classe.

Lorsqu'il y a trop de méthodes statiques dans le modèle et qu'il manque beaucoup de méthodes d'instances, on constate que les contrôleurs ont tendance à passer de multiples objets aux vues. Avec des méthodes d'instance, on peut éviter ce problème. Par exemple, si la classe `User` possède une méthode d'instance `get_my_items()` (qui délègue éventuellement à une méthode statique de recherche de la classe `Item`), alors la page qui affiche la liste des annonces n'a besoin que de connaître l'utilisateur connecté. Dans la vue en question, on appellera `$user->get_my_items()` pour obtenir cette liste. Dans le même ordre d'idée, on devrait pouvoir demander à une annonce la liste de ses offres, ...

Règles conseillées

- Chaque classe devrait avoir une méthode d'instance `validate()` qui effectue des validations métier (éventuellement déléguées à des méthodes privées) et retourne un tableau de messages d'erreurs (éventuellement vide).
- Chaque classe devrait avoir une méthode d'instance `persist()` qui provoque la persistance en base de données de l'instance courante. Cette méthode permet de vérifier les données de l'instance (par exemple en faisant appel à `validate()`) et de ne procéder à la mise-à-jour en DB que s'il n'y a pas d'erreurs. S'il y a des erreurs, elles sont retournées sous la forme d'un tableau de messages d'erreur. Cette méthode s'occupe aussi de mettre à jour les attributs correspondants aux clés primaires auto-incrémentées.
- Chaque classe devrait avoir une méthode d'instance `delete()` qui permet de supprimer le record correspondant en base de données.
- Chaque classe du modèle devrait avoir un constructeur qui prend en paramètre des valeurs pour les différents attributs. Les données non requises par la base de données ont des valeurs par défaut (colonnes ayant une valeur par défaut, dont les colonnes *nullables*, ou clés auto-générées). Les données qui correspondent à des clés étrangères sont traitées de manière particulière (voir point suivant).
- Si un objet ne peut pas exister sans appartenir à un autre objet (composition), alors l'objet "conteneur" peut être passé comme argument du constructeur de l'objet "contenu". Cet argument sera alors utilisé pour mettre à jour l’attribut correspondant à la clé étrangère.
- Dans la mesure du possible, on évitera les méthodes qui prennent en paramètre une ou plusieurs données d'une instance. On préférera, dans ce cas, passer l'instance comme seul paramètre. Exceptions : le constructeur et les méthodes statiques de recherche.
- Les attributs correspondants aux clés étrangères devraient être des attributs privés. Ils ne sont utilisés que par les méthodes qui font des requêtes en base de données. Par contre, on prévoira une méthode publique qui retourne l'instance de la classe visée par la clé étrangère.